



PÁZMÁNY

Pázmány Péter Catholic University

Faculty of Information Technology and Bionics

ÖNÁLLÓ LABORATÓRIUM

Kétdimenziós és háromdimenziós sakkváltozatok tervezése és programozása

KOVÁCS Domonkos

Mérnökinformatikus Bsc

Témavezető: Dr. Fogarasi Norbert

2026

Tartalomjegyzék

1. Bevezetés	4
1.1. Feladatom	4
2. Előzmények	5
2.1. A fullerén alakzat topológiája és gráfelméleti modellje	5
2.2. A fullerén sakk szabályrendszere	7
2.2.1. Alapvető mozgástípusok és irányok	7
2.2.2. Bástyá mozgása	7
2.2.3. Futó mozgása	8
2.2.4. Zászló és sáncolás	9
2.2.5. Kezdőállás	9
2.2.6. A játék menete	10
3. Tervezés	11
3.1. Tábla tervezése	11
3.1.1. Szabályok és felismerések	11
3.1.2. Mező kapcsolati nehézségek	11
3.1.3. Mező hálózat felépítése	13
3.1.4. Csúcshomszédshág kiépítése	15
3.2. A bábhk osztályszerkezete és mozgási logikája	16
3.2.1. A bástyá mozgási logikájának implementációja	16
4. Eredmények	19
4.1. Átfogó mozgástér-ellenőrzés (Unit Teszt)	19
4.2. Kezdőállás szabályos lépéseinek validációja	20
4.3. Random játszma szimuláció (End-to-End teszt)	21
4.4. Fizikális ellenőrzés	21
5. Összefoglalás	23

1. fejezet

Bevezetés

A háromdimenziós és nem-hagyományos felületeken történő játékmodellezés célja, hogy a klasszikus, síkbeli játékszabályokat kiterjessze a nem-euklideszi geometriák és a komplex topológiák világába. Egy ilyen szoftveres rendszer feladatai közé tartozik a játéktér matematikai reprezentációja, a bábuk mozgásterének precíz és szabályos kiszámítása a görbült felületen, valamint a lépések validálása. A rendszernek képesnek kell lennie a diszkrét mezők közötti kapcsolatok statikus és konstans idejű kezelésére, az egyedi topológiai interakciók levezetésére, illetve a játékállapot folyamatos nyomon követésére.

1.1. Feladatom

Önálló laboratóriumi munkám során a 72 mezős fullerén sakk játék szoftveres rendszerének részleges megvalósításában vettem részt, amelynek célja a háromdimenziós játéktér modellezése, valamint a bábuk szabályszerű működésének biztosítása. Ezen önálló laboratóriumi dolgozat alapját Kiss Márton Zsoltnak a fullerén alakzatra történő sakk implementáció ötlete adta.

Feladatom a játéktér hálózati topológiájának kiépítése, a bábuk mozgási és ütközésvizsgálati logikájának szoftveres implementálása volt. A játékmotor megalkotásával közvetlenül támogatom a projekt grafikai megvalósítását, valamint a játék tanítási folyamatát is, amik teljes mértékben erre a logikára támaszkodnak.

2. fejezet

Előzmények

2.1. A fullerén alakzat topológiája és gráfelméleti modellje

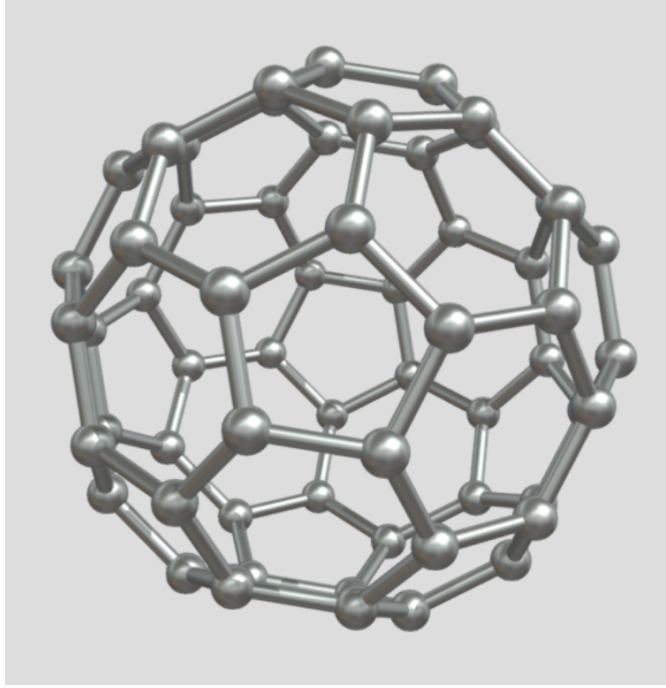
A háromdimenziós játéktér matematikai hátterét a szilárdtestfizika és a szerkezeti kémiából ismert fullerén-topológia biztosítja. A fullerének olyan zárt felülettel rendelkező konvex poliéderek, amelyeket kizárólag n -szögek – a leggyakoribb esetekben ötszögek és hatszögek – alkotnak. Az ilyen molekuláris szerkezetek térbeli záródását és geometriai stabilitását az Euler-féle poliédertétel szabályozza, amely a csúcsok (V), az élek (E) és a lapok (F) száma közötti összefüggést az alábbi formában írja le:

$$V - E + F = 2 \quad (2.1)$$

Mivel a fullerénszerkezetekben minden csúcsból pontosan három él indul ki (3-reguláris gráf), a fenti topológiai egyenletből matematikailag egyenesen következik, hogy egy gömbszerűen lezáródó test kialakításához a hatszögek számától teljesen függetlenül pontosan 12 darab ötszög megléte szükséges [4].

A képen 2.1 egy klasszikus C_{60} fullerén molekula (csonkított ikozaéder) látható [5], amely az Euler-féle poliédertétel kényszere ($V - E + F = 2$) és a 3-regularitás miatt a Kiss Márton Zsolt (1.1) által definiált játéktér közvetlen matematikai alapja [4]. A fundamentális különbség a méretezésben és az absztrakcióban rejlik: míg a C_{60} molekula 20 hatszöget tartalmaz, addig a játékhoz használt C_{60+12n} fulleréncsalád $n = 1$ állapotának felel meg.

Geometriailag pontosan 12 ötszöget és 60 hatszöget tartalmaz. Egy ilyen görbült, nem-euklideszi felületen a klasszikus síkbeli sakktáblák kétdimenziós koordináta-rendszere (például a mátrixalapú indexelés) nem alkalmazható. Emiatt a játéktér architektúráját egy molekuláris gráfként lehet leírni, ahol a mezők alkotják a gráf csúcsait, a köztük lévő



2.1. ábra. C_{60} fullerén molekula [5]

fizikai határok és szomszédságok pedig a gráf éleit határozzák meg [2].

A megvalósítás során ez a 3-reguláris síkbeli gráf kerül leképezésre statikus adatszerkezetekbe a játékmotor inicializálásakor. A bábuk mozgás- és ütközésvizsgálati algoritmusai, a bástya lapszomszédsági és a futó csúcsszomszédsági lépései. Így nem geometriai koordinátákat vagy távolságokat számolnak, hanem ezen a fix topológiai hálózaton és annak él-tulajdonságaira épülnek. Ez a gráfelméleti megközelítés biztosítja, hogy a játékszabályok validálása determinisztikus és konstans idejű legyen, teljesen függetlenül attól, hogy a későbbi grafikus felület, hogyan vetíti ki a háromdimenziós gömbfelületet a kijelzőn. [2].

A saktábla hagyományos kétdimenziós, euklideszi geometriájának felülbírálása és nem hagyományos felületekre történő kiterjesztése a játékelméleti kutatások egy régóta dokumentált ága.

D. B. Pritchard a sakkvariánsok osztályozásáról szóló művében rávilágít arra, hogy a játéktér topológiájának megváltoztatása alapjaiban formálja át a bábuk taktikai értékét és a lépésvonalak matematikai tulajdonságait [3].

A fullerén sakk esetében alkalmazott topológia ebből a szempontból egy radikális lépés: a zárt, hat- és ötszögekből álló görbült térháló megszünteti a klasszikus értelemben vett táblaszéleket, így a bábuk mozgástere önmagába záródik.

2.2. A fullerén sakk szabályrendszere

A fullerén sakk két bábut vett át a klasszikus sakkból, és egy új zászló bábuval egészíti ki. Így három bábu lett definiálva, amiknek viszont a mozgása eltér a hagyományos sakkétól a fullerén alakzat miatt. A 72 mezős csonkított ikozaéder felülete teljesen új mozgási karakterisztikát és taktikai elemeket követel meg. A síkbeli sakktáblákkal ellentétben ezen a görbült térhálón nincsenek fix táblaszélek, a felület önmagába záródik, a mezők pedig nem négyzetek, hanem 12 darab ötszög és 60 darab hatszög mezőből állnak. Ez a topológiai sajátosság alapjaiban határozza meg a bábuk lépési és ütközési szabályait.

2.2.1. Alapvető mozgástípusok és irányok

A bábuk mozgása a játéktérként szolgáló gráf élei és csúcsai mentén van definiálva, elválasztva egymástól a lap- és a csúcsszomszédságon alapuló haladást.

A hatszögletű cellákon értelmezett lépési irányok elméleti alapjai Władysław Gliniski 1936-os hexagonális sakkjára vezethetők vissza [1]. Gliniski rendszerében jelent meg először tisztán a sejt-alapú topológiai megközelítés, ahol a klasszikus „sor” és „oszlop” fogalmakat a lap- (ortogonális) és csúcsszomszédsági (átlós) (2.2.1) haladás váltotta fel.

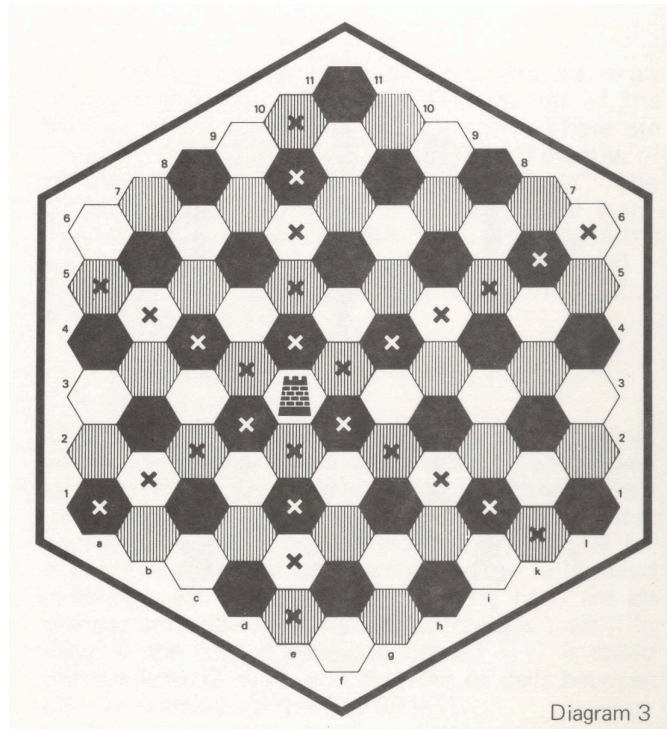
A bástya-típusú mozgás a közös éleken keresztüli folytonos haladást jelenti, míg a futó-típusú lépések a cellák közös csúcspontjain keresztül valósul meg [3].

- **Lapszomszédsági (ortogonális) mozgás:** A lépés során a bábu olyan mezőre lép át, amellyel a kiindulási mezője egy közös élben (határvonalban) osztozik. Ez a mozgás felel meg a klasszikus sakk egyenes vonalú (bástya-típusú) lépéseinek.
- **Csúcsszomszédsági (átlós) mozgás:** A lépés során a bábu olyan mezőre érkezik, amellyel a kiindulási mezője csak egyetlen közös csúcspontban érintkezik, de nincs közös élük. Ez a mozgás felel meg a klasszikus sakk átlós (futó-típusú) lépéseinek.

2.2.2. Bástya mozgása

A bástya kizárólag lapszomszédság (2.2.1) mentén haladhat, egyenes vonalban, tetszőleges távolságra, amíg akadályba (saját vagy ellenséges bábuba) nem ütközik. A hatszögekből álló területeken az „egyenes irány” egyértelmű: a szemközti élen keresztül történő kilépés biztosítja a folytonosságot.

Amikor azonban a bástya útja egy ötszögű mezőt keresztez, a szemközti él fogalma megváltozik, mivel az ötszög páratlan számú éllel rendelkezik. Az ötszögre lépve a bástya



2.2. ábra. Bástya mozgásának szemléltetése [1]

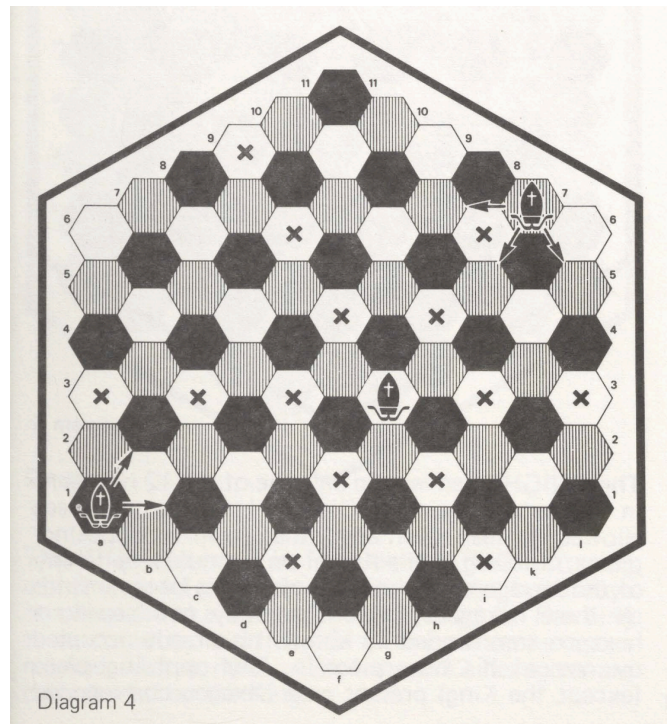
nem haladhat tovább egyenesen anélkül, hogy ne törné meg a szimmetriát. A szabályzat szerint az ötszög elhagyásakor a bástya útja kötelezően megtörik (irányt vált), ami egyfajta természetes görbületet visz a lépésvonalba. Tehát, ha a bástya egy ötszögbe ütközik, megáll az ötszögön, és csak a következő körben haladhat tovább.

2.2.3. Futó mozgása

A futó kizárólag csúcsszomszédság (2.2.1) mentén, azaz átlósan közlekedhet, amíg akadályba (saját vagy ellenséges bábuba) nem ütközik.

A futó mozgását is megtöri egy ötszögű mező, ha az útjába kerül. Ekkor, az ellenfél következik és csak a következő körben tud újra lépni az ötszögről. Ez esetben a pálya geometriája miatt mindenképp irányt vált az előző lépéshez képest.

A futó mozgásának van még egy kritikus szabálya. A szabály szerint azon a csúcson nem haladhat át a futó, ahol két ellenséges bábú között menne át. Ekkor abban az irányban megtörik a futó mozgása. Azonban a szabály engedi az egy ellenséges és egy saját bábú, illetve két saját bábú közötti haladást.



2.3. ábra. Futó mozgásának szemléltetése [1]

2.2.4. Zászló és sáncolás

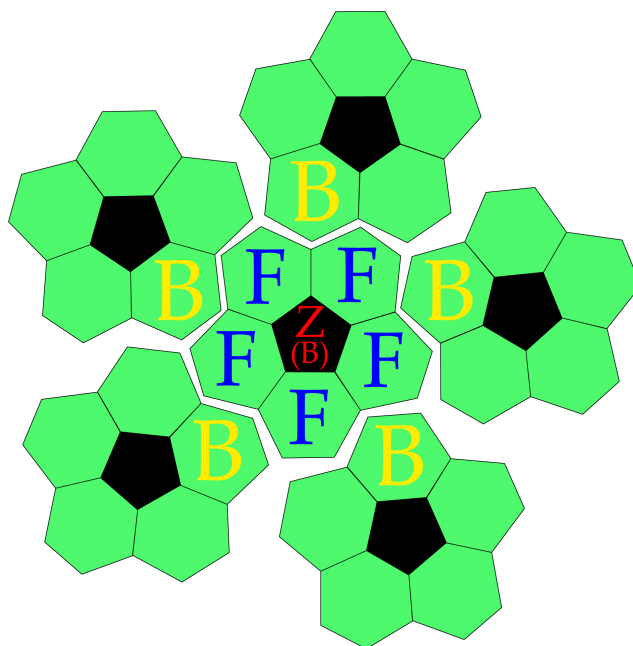
A zászló a legfontosabb bábú a játékban, hiszen őt kell megvédenünk (2.2.6). A zászló nem tud lépni, azonban van egy hasonló mozgása, mint a hagyományos sakokban, amit itt is sáncolásnak nevezünk. A zászló tárol egy virtuális információt magáról, hogy éppen bástya vagy futó típusa van.

Amikor sáncolunk a zászlóval, akkor kizárólag egy lapszomszédjával tud "helyet" cserélni akkor és csak akkor, ha a lapszomszédja más típusú bábú, mint a zászló virtuálisan tárolt típusa. A tényleges helycsere helyett viszont csak a típusaik cserélődnek ki. A sáncolás egy lépésnek számít, viszont egy játszma alatt bármennyiszer lehet sáncolni.

Ez a megoldás hasonlít a Pritchard által vizsgált bábu-csere (Piece-Changing) sakkvariánsokkal [3]. Ezen történelmi játékok sajátossága, hogy a bábuk nem rendelkeznek állandó karakterisztikával, hanem lépés közben vagy bizonyos kitüntetett mezőket elérve megváltoztatják a típusukat és mozgási képességeiket.

2.2.5. Kezdőállás

A test szimmetriájából adódik, hogy minden ötszögnek van egy párja pont a test másik oldalán vele szemben. Esetünkben ez az 1-es és 67-es mezők (3.1, 3.2). Ezen a két, egymással szemben lévő mezőn helyezkedik el a két csapat zászlaja.



2.4. ábra. Általános kezdőállás

A zászlók lapszomszédain futók állnak, míg a csúcsszomszédain bástyák. A zászlónak a játék kezdetekor bástya típusa van. (2.4)

2.2.6. A játék menete

Az egyik csapat világos, míg a másik csapat sötét bábúkkal rendelkezik, akár a hagyományos sakokban. A világos csapat lép először, és inntől felváltva lépnek a játékosok. A lépés egy adott bábúval egy üres mezőre, vagy egy olyan mezőre való lépést jelent ahol az ellefél egyik bábúja tartózkodik. Illetve lépésnek számít egy sáncolás is (2.2.4).

A játék lényege, hogy az egyik csapat leüsse az ellenfél zászlaját. Ekkor a játék véget ér.

3. fejezet

Tervezés

3.1. Tábla tervezése

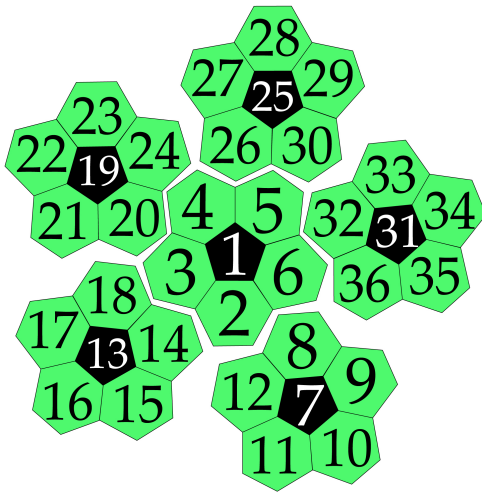
3.1.1. Szabályok és felismerések

A munkát rajzolással kezdtem, hogy el tudjam képzelni, hogyan épül fel a fullerén rács. Hosszas ötletelés után megalkottam a mezők számozásának logikáját. Ehhez lefektettem három alapszabályt, amit részben én találtam ki, részben a fullerén alakzat adottságaiból következtettem ki. Ezek alapján alakítottam ki a végleges számozott mezőket: (3.1, 3.2)

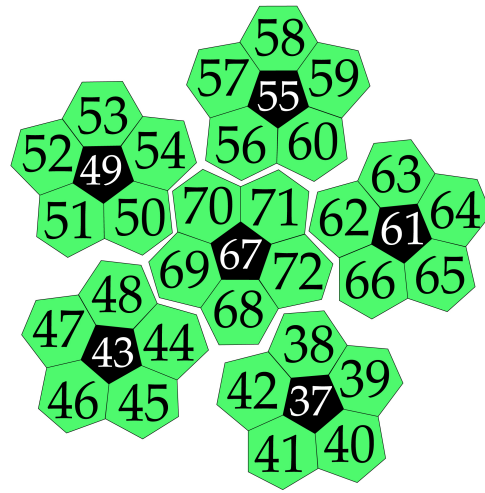
- **Szemközti párok szabálya:** A megadott párok (1-67, 7-61, 13-55, 19-49, 25-43, 31-37) a fullerén átellenes pontjai, így ezek környezete soha nem érintkezhet közvetlenül. Sőt, a pár egyik feléből a másik felébe két ötszögnyi út vezet.
- **Két-hatszög szabály:** A fő blokkok (1-67) körüli ötszögek hatszögei közül a legkisebb és legnagyobb elemek kapcsolódnak a fő blokk hatszögeihez (pl. a 7-es blokk 8-as és 12-es mezője (3.1)). Minden ötszögnek tudjuk milyen sorrendben és merre vannak az elemei, mert az óramutató járásának megfelelően helyezkednek el.
- **Hármas csomópont szabály:** Minden hatszöghöz három ötszög hatszögei adják a lapszomszédait, és ebből csak az egyik ad egyet. Ez a mező lesz az ami a másik két ötszög hatszögeihez kapcsolódik és fogja össze őket (pl. a 13-as blokk 14-es mezője a 2-es és 12-es mezőkhöz (3.1)).

3.1.2. Mező kapcsolati nehézségek

Az első verziókban még egyszerű `if` feltételekkel próbáltam összekötni a blokkokat. A tesztelés során azonban észrevettem, hogy bizonyos hatszögeknek 7 vagy több szomszédja



3.1. ábra. Az alakzat végleges számozott mezőinek felső fele



3.2. ábra. Az alakzat végleges számozott mezőinek alsó fele

lett, ami geometriailag lehetetlen.

Ekkor jöttem rá, hogy az ötszög-központú blokkok (pl. 1-es blokk és 7-es blokk) közötti varratokat nem lehet véletlenszerűen kezelni. Bevezettem a `pLinks` táblázatot (14), amely az ötszögek globális elrendezését tárolja.

A `pLinks[1]` és `pLinks[67]` adottak olyan szempontból, hogy növekvő sorrendben, óramutató járásának megegyezően következnek a blokkok. Az összes többinél, attól függ, hogy az 1-es vagy 67-es blokk-hoz tartoznak. Amelyik az 1-es blokkhoz tartoznak, azokat mindig az 1-es blokkhoz igazítjuk és onnan óramutató járásnak megfelelően következik a többi blokk. Ugyanígy van a 67-es blokk körülöttiekkel.

```
std::map<int, std::vector<int>> pLinks;
pLinks[1] = {7, 13, 19, 25, 31};
pLinks[7] = {1, 31, 49, 43, 13};
pLinks[13] = {1, 7, 43, 37, 19};
pLinks[19] = {1, 13, 37, 61, 25};
pLinks[25] = {1, 19, 61, 55, 31};
pLinks[31] = {1, 25, 55, 49, 7};
pLinks[67] = {37, 43, 49, 55, 61};
pLinks[37] = {67, 61, 19, 13, 43};
pLinks[43] = {67, 37, 13, 7, 49};
pLinks[49] = {67, 43, 7, 31, 55};
pLinks[55] = {67, 49, 31, 25, 61};
```

```
pLinks[61] = {67, 55, 25, 19, 37};
```

Listing 1. pLinks táblázat

Erre nagyon nagy szükség volt, hiszen így, hogy már az ötszögek elhelyezkedését tudjuk egymáshoz képest, fel tudjuk tölteni a köztük lévő hézagokat hatszögekkel.

3.1.3. Mező hálózat felépítése

A hálózat felépítése két for ciklusból áll. Az egyik 1-től megy 67-ig és 6-osával lépked, így végigmegy az összes ötszögen. Ezen belül a másik for ciklus 0-tól 5-ig megy és egyesével lépked az adott ötszög lapszomszédjain.

A belső ciklus elején sok mindent kiszámolok előre, hogy később ezekkel könnyebb legyen számolni:

```
int hex = p + 1 + i;  
int nextHex = p + 1 + (i + 1) % 5;  
int prevHex = p + 1 + (i + 4) % 5;  
int targetP = pLinks[p][i];  
int nextTargetP = pLinks[p][(i + 1) % 5];
```

Listing 2. Változók kiszámolása

A **hex** az adott mező száma, a **nextHex** és **prevHex** az anya ötszögnek a következő és előző eleme óramutató járásnak megfelelően. A **targetP** változó annak az ötszögnek a számát tárolja, amelyiknek két hatszöge csatlakozik, amíg a **nextTargetP** annak az ötszögnek a számát tárolja, amelyiknek egy hatszöge csatlakozik a **hex** vizsgált mezőhöz.

Ahogy végigmegy a külső ötszögeken lépkedő for ciklus, mindig hozzárendeli lapszomszédként az éppen vizsgált, körülötte lévő hatszöget. Így az ötszögek körüli mezők szépen növekvő sorrendben következnek.

Az egyes hatszögek lapszomszédainak óramutató járásával megegyező indexeléséhez egyesével beállítjuk a belső for ciklusban, hogy az adott mező lapszomszédjainak melyik indexére melyik mező kerüljön.

Az első két indexen lévő lapszomszéd már adott, hiszen az első lapszomszéd (0. index) minden esetben az "anya" ötszög (p), ami az az ötszög, amivel tudjuk, hogy lapszomszéd. A második lapszomszéd (1. index) óramutató járásával megegyezően a már kiszámolt **prevHex**, ami az "anya" ötszögben vizsgált mező előtti elemet adja vissza (óramutató járásnak ellentétes irányban az első elem).

```
tiles[hex].edgeNeighbors[0] = p;
```

```
tiles[hex].edgeNeighbors[1] = prevHex;
```

Listing 3. 1. és 2. lapszomszéd hozzárendelése

A következő két lapszomszéd megállapításához az egyik korábbi alapszabályom (3.1.1) szerint járok el, ami kimondja, hogy az 1-es és 67-es ötszögek körüli hatszögekhez a fő (1, 67) ötszögek körüli hatszögek legkisebb és legnagyobb számú hatszögei adják a lapszomszédait. Az érintett hatszögeknek a lapszomszédai ez alapján épülnek fel, a többi hatszög meg "alkalmazkodik" a szabályhoz.

A **targetP** ötszögnek kiszámoljuk melyik két mezője csatlakozik, majd hozzárendeljük a vizsgált mező 3. és 4. (2. és 3. indexű) lapszomszédjának.

```
int j = -1;
for(int k=0; k<5; ++k) {
    if(pLinks[targetP][k] == p) {
        j = k; //j = 0
    }
}
tiles[hex].edgeNeighbors[2] = targetP + 1 + j;
tiles[hex].edgeNeighbors[3] = targetP + 1 + (j + 4) % 5;
```

Listing 4. 3. és 4. lapszomszéd hozzárendelése

A következő lapszomszéd megadásához a hármas csomópont szabályhoz (3.1.1) kell visszanyúlni. A szabály szerint minden hatszöghöz tartozik egy olyan hatszög ami nem az "anya", és nem is a két lapszomszédot adó ötszöghöz tartozik. Ekkor lép be egy harmadik ötszöghöz tartozó mező, ami csak ezt az egy lapszomszédot adja. A már kiszámolt **nextTargetP** változó adja meg melyik ötszög fogja adni azt a mezőt. Utána megkeresünk hanyadik indexű eleme lesz ami csatlakozik és hozzárendeljük a vizsgált mező 5. (4. index) lapszomszédjának.

```
int k_idx = -1;
for(int k=0; k<5; ++k) {
    if(pLinks[nextTargetP][k] == p) {
        k_idx = k;
    }
}
tiles[hex].edgeNeighbors[4] = nextTargetP + 1 + k_idx;
```

Listing 5. 5. lapszomszéd hozzárendelése

Az utolsó elem már szintén ki lett számolva, hiszen ez a `nextHex` változóban tárolt szám, ami az "anya" ötszögben vizsgált mező utáni elemet adja vissza (óramutató járásnak megegyező irányba az első elem).

```
tiles[hex].edgeNeighbors[5] = nextHex;
```

Listing 6. 6. lapszomszéd hozzárendelése

3.1.4. Csúcsszomszédság kiépítése

Mivel már tudjuk az egyes mezők lapszomszédait, ezért könnyen ki tudjuk számolni a hozzá tartozó csúcsszomszédokat is. A végleges csúcsszomszédsági algoritmust a hálózat-építő két for ciklus után kell futtatni, hiszen ekkor már tudjuk, hogy az egyes mezők hol helyezkednek el a táblán.

Az alap logikám innentől az volt, hogy az éppen vizsgált mezőnek az lehet csúcsszomszédja, amivel van két közös lapszomszédjuk. Ehhez a logikához négy for ciklust kell használnom, az első végigmegy az összes mezőn 1-től 72-ig. Ahhoz, hogy ne tudjak túlindexelni, és hogy univerzális legyen öt illetve hatszögekre is, mindig elmentem az adott mező lapszomszédsági vektorának a méretét. Ebből kiderül, hogy éppen öt vagy hatszöget vizsgálunk.

```
int pSize = tiles[p].edgeNeighbors.size();
```

Listing 7. 6. lapszomszéd hozzárendelése

Az újabb for ciklusban nullától a `pSize`-ig indexelünk, és ezen belül nevezzük meg a vizsgált elem két lapszomszédját aminek majd összevetjük a lapszomszédjait, hogy mi az egyezés. Ahhoz, hogy ne indexeljünk túl, nem csak egy szimpla eltolást alkalmazok, hanem a `pSize`-al maradékos osztok. Az `n1` és `n2`-nek is létrehozok egy `size` változót `n1Size` és `n2Size` néven, a későbbi indexelés miatt.

```
int n1 = tiles[p].edgeNeighbors[k];
int n2 = tiles[p].edgeNeighbors[(k + 1) % pSize];
int n1Size = tiles[n1].edgeNeighbors.size();
int n2Size = tiles[n2].edgeNeighbors.size();
```

Listing 8. Csúcsszomszéd felé lévő két lapszomszéd meghatározása

A következő dupla for ciklusban, összevetem az `n1` és `n2`-nek a lapszomszédait. Amikor, van egyezés és nem a vizsgált mező az, akkor azt beállítom a soron következő csúcsszomszédjának.

```

for (int i = 0; i < n1Size; ++i) {
for (int j = 0; j < n2Size; ++j) {
    if (tiles[n1].edgeNeighbors[i] != -1 &&
        tiles[n1].edgeNeighbors[i] != p &&
        tiles[n1].edgeNeighbors[i] == tiles[n2].edgeNeighbors[j]){
        tiles[p].vertexNeighbors[k] = tiles[n1].edgeNeighbors[i];
    }
}
}

```

Listing 9. Csúcsszomszéd megtalálása

3.2. A bábúk osztályszerkezete és mozgási logikája

A játékmotor objektumorientált architektúrájában a bábuk reprezentációjáért és egyedi viselkedéséért egy önálló **Piece** osztály felel. Minden egyes bábú példányosításakor rögzítésre kerül annak aktuális pozíciója (**currentTileId**), a csapata (**Team**), valamint az alapértelmezett típusa (**PieceType**), amely bástya (**Rook**), futó (**bishop**) vagy zászló (**Flag**) lehet.

3.2.1. A bástya mozgási logikájának implementációja

A bástya mozgásterének kiszámítását a **getRookMoves()** metódus végzi. Az algoritmus a bábú kiinduló mezőjének lapszomszédosági vektorából (**edgeNeighbors**) indul ki, és minden elérhető irányban egy **while** ciklus segítségével lépked előre a szomszédos hatszögeken. A folytonos, egyenes vonalú haladást a **getOppositeNeighbor()** segédfüggvény garantálja, amely a fix óramutató szerinti indexkiosztást kihasználva a $(\text{belépési_index} + 3) \% 6$ képlettel határozza meg a szemközti élen található következő mezőt.

Az algoritmus két ponton szakítja meg a keresést az adott irányban: ha akadályba ütközik, vagy ha elér egy ötszöget. Mivel az ötszög típusú mezőkhöz nincs szemközti él definiálva (a függvény **-1**-gyel tér vissza), a bástya mozgása az ötszögre lépve azonnal megszakad, biztosítva a szabályrendszerben (2.2.2) leírt mozgáskorlátozást.

A futó mozgási logikájának implementációja

A futó átlós haladási mechanizmusát a `getBishopMoves()` függvény valósítja meg, amely a `vertexNeighbors` vektort használja fel. A mozgásirányok feltérképezése során az algoritmusnak minden egyes lépésnél validálnia kell, hogy valóban minden szabály (2.2.3) szerint haladhat abban az irányban.

Ehhez a program lekéri az átlós lépésvonal két oldalán elhelyezkedő lapszomszédos mezők azonosítóit. Amennyiben mindkét lapszomszédos mező foglalt, és a rajtuk álló bábuk az ellenfél csapatához tartoznak, a csomópont lezárul, és a ciklus megszakítja a futó további haladását abban az irányban.

Ha ez a probléma nem áll fenn, a futó mindaddig haladhat a csúcsok mentén, amíg egy másik bábu vagy egy ötszögbe nem ütközik, amely a bástyához hasonlóan a futó mozgását is blokkolja.

A zászló sáncolási logikájának implementációja

A zászló mint kitüntetett, védendő bábu nem rendelkezik önálló, hagyományos mozgástérrel, így helyváltoztatásra kizárólag a sáncolás (helycsere) mechanizmusán keresztül képes. A kódban ezt a `getFlagSwaps()` függvény végzi. A zászló egy belső állapotjelzőt (`flagState`) tart fenn, amely meghatározza, hogy a bábu virtuálisan bástya vagy futó üzemmódban van-e. Sáncolás kezdeményezésekor a program átvizsgálja a közvetlen lapszomszédokat, és olyan saját bábukat keres, amelyek típusa eltér a zászló belső állapotától (bástya mód esetén futót, futó mód esetén bástyát keres).

A helycsere tényleges végrehajtását az `executeFlagSwap()` függvény csinálja, amely a memóriában megcseréli a pozíciókat, és ezzel egy időben átírja a bábuk típusát is. A zászló felveszi a kicserélendő bábú fizikai típusát, a bábú pedig az új zászlóvá válik, miközben a belső `flagState` állapotjelzőt átállítom az ellenkező értékre.

Lépések validálása és ütközésvizsgálat

A játékmenet integritását és a szabályok szigorú betartatását a `GameEngine` és a `Piece` osztályok együttműködése biztosítja az ütközésvizsgálati fázisban. Amikor meghívom a `processMove(onnan, hova)` függvényt, a rendszer első lépésként ellenőrzi, hogy a kiválasztott mezőn létezik-e bábu, és az a soron következő játékos csapatához tartozik-e.

Ezt követően a `canMoveTo()` függvény segítségével lefut a célmező validációja, amely az alábbi ütközési szabályokat érvényesíti:

- **Üres mező:** Ha a célmező üres, a bábu akadálytalanul ráléphet, amennyiben az adott bábu típusára vonatkozó szabályok érvényesnek ítélik.
- **Ellenséges bábu (Ütés):** Ha a célmezőn ellenséges bábu áll, a mozgás validnak minősül, az útvonalkereső ciklusok itt véget érnek, a `executeMoveLogic()` pedig törli az ellenfél bábuját a tábláról. Amennyiben a leütött bábu egy zászló, a `gameOver` zászló igazra változik, és a játék véget ér.
- **Saját bábu:** Ha a célmezőn saját bábu áll, a mozgás alapesetben tiltott, kivéve, ha az interakció egy szabályos zászló-cserét (sáncolást) valósít meg, mert ekkor a normál lépés helyett a pozíciók és típusok cseréje fut le.

Ezek a szabályok és validációs folyamatok garantálják, hogy játék konzisztens legyen, és kizárólag a fullerén rács topológiájának megfelelő, szabályos lépések hajtódjanak végre.

4. fejezet

Eredmények

A megvalósított játékmotor validálása és a szabályrendszer pontosságának ellenőrzése érdekében három átfogó tesztet csináltam. Mivel a fullerén sakk játéktér geometriája és topológiája rendkívül komplex, így a működést nemcsak automatizált adatelemzéssel, hanem a fizikai, 3D nyomtatott alakzaton végzett manuális lépéskövetéssel is ellenőriztem.

A tesztelési folyamatot három fő fázisra bontottam: izolált egységtesztekre (Unit Test), kezdőállás-validációra, valamint teljes játékmenetet lefedő end-to-end szimulációra.

4.1. Átfogó mozgástér-ellenőrzés (Unit Teszt)

Az ellenőrzés első fázisában a `UnitTest()` függvényben egy izolált környezetben vizsgáltam meg a bástya és a futó elméleti mozgásterét. Az algoritmus egy teljesen üres táblán, egymás után mind a 72 mezőre lehelyezte a bábukat, majd legenerálta az onnan elérhető összes elméleti célmezőt.

Ez a teszt kritikus fontosságú volt a hálózati topológia helyességének igazolásában. A generált kimenetek elemzése során bebizonyosodott, hogy:

- A bástya mozgástere hatszögletű mezőkről indítva egyenes vonalban halad, míg ötszögű mezőre érkezve a mozgásvonal azonnal és szabályszerűen megszakad, mivel az ötszögnek a motor logikájában nincs szemközti éle (2.2.2).
- A futó lépéstávolságai pontosan követik a csúcsszomszédsági láncolatot, és az ötszögek az átlós haladást is blokkolják a szabályrendszernek megfelelően (2.2.3).

```

Szabalyos lepesek a kezdopoziciobol (FEHER):
Babu: FLAG Mezon: 1 -> Lehetseges celpontok: 2 3 4 5 6
Babu: BISHOP Mezon: 2 -> Lehetseges celpontok: 7 15 18 34 36 38 40 45 55 68
Babu: BISHOP Mezon: 3 -> Lehetseges celpontok: 10 12 13 21 24 39 49 62 64 72
Babu: BISHOP Mezon: 4 -> Lehetseges celpontok: 16 18 19 27 30 43 56 58 63 71
Babu: BISHOP Mezon: 5 -> Lehetseges celpontok: 22 24 25 33 36 37 50 52 57 70
Babu: BISHOP Mezon: 6 -> Lehetseges celpontok: 9 12 28 30 31 44 46 51 61 69
Babu: ROOK Mezon: 8 -> Lehetseges celpontok: 7 9 12 15 16 31 36 37 41 49 52
Babu: ROOK Mezon: 14 -> Lehetseges celpontok: 7 12 13 15 18 21 22 43 46 61 65
Babu: ROOK Mezon: 20 -> Lehetseges celpontok: 13 18 19 21 24 27 28 37 40 55 59
Babu: ROOK Mezon: 26 -> Lehetseges celpontok: 19 24 25 27 30 33 34 49 53 61 64
Babu: ROOK Mezon: 32 -> Lehetseges celpontok: 9 10 25 30 31 33 36 43 47 55 58

```

4.1. ábra. `teszt_kezdo_feher.txt` tartalma

```

Szabalyos lepesek a kezdopoziciobol (FEKETE):
Babu: ROOK Mezon: 38 -> Lehetseges celpontok: 7 11 19 22 37 39 42 45 46 61 66
Babu: ROOK Mezon: 44 -> Lehetseges celpontok: 13 16 31 35 37 42 43 45 48 51 52
Babu: ROOK Mezon: 50 -> Lehetseges celpontok: 7 10 25 29 43 48 49 51 54 57 58
Babu: ROOK Mezon: 56 -> Lehetseges celpontok: 19 23 31 34 49 54 55 57 60 63 64
Babu: ROOK Mezon: 62 -> Lehetseges celpontok: 13 17 25 28 39 40 55 60 61 63 66
Babu: FLAG Mezon: 67 -> Lehetseges celpontok: 68 69 70 71 72
Babu: BISHOP Mezon: 68 -> Lehetseges celpontok: 2 8 10 15 25 37 45 48 64 66
Babu: BISHOP Mezon: 69 -> Lehetseges celpontok: 6 9 19 32 34 40 42 43 51 54
Babu: BISHOP Mezon: 70 -> Lehetseges celpontok: 5 13 26 28 33 46 48 49 57 60
Babu: BISHOP Mezon: 71 -> Lehetseges celpontok: 4 7 20 22 27 52 54 55 63 66
Babu: BISHOP Mezon: 72 -> Lehetseges celpontok: 3 14 16 21 31 39 42 58 60 61

```

4.2. ábra. `teszt_kezdo_fekete.txt`

4.2. Kezdőállás szabályos lépéseinek validációja

A második fázisban a `InitialStateTest()` függvényben a játék hivatalos kezdőfelállítását (2.4) vizsgáltam meg mindkét csapat (világos és sötét) szempontjából. A teszt célja annak ellenőrzése volt, hogy a sűrű kiindulási pozícióban helyesen veszi-e figyelembe a saját és ellenséges bábuk jelenlétét, az ütközésvizsgálatot és a speciális sáncolási (zászló-csere) opciókat.

A generált kimeneti fájlok (`teszt_kezdo_feher.txt` (4.1) és `teszt_kezdo_fekete.txt` (4.2)) elemzése az alábbi eredményeket hozta:

- **Zászló interakciók:** Az 1-es mezőn álló fehér zászló számára pontosan a körülötte elhelyezkedő 2, 3, 4, 5, 6-os mezőket dobta ki lehetséges célpontként. Mivel a játék kezdetén a zászló bástya módban van, a lapszomszédos mezőkön álló saját futókkal (`bishop`) képes végrehajtani a cserét, ami igazolja a sáncolási logika helyességét. Ugyanez a zárt, 5 elérhető lépésből álló lista jelent meg a sötét csapat esetében is a 67-es mezőről indulva.
- **Bástyák és futók mozgástere:** A bástyák és futók esetében helyesen mutatta

```

TELJES RANDOM JATSZMA SZIMULACIO

1. lépés: FEHER [ROOK] 26 -> 27
2. lépés: FEKETE [BISHOP] 72 -> 39
3. lépés: FEHER [ROOK] 8 -> 7
4. lépés: FEKETE [BISHOP] 71 -> 55
5. lépés: FEHER [ROOK] 27 -> 23
6. lépés: FEKETE [ROOK] 56 -> 72
7. lépés: FEHER [BISHOP] 4 -> 43
8. lépés: FEKETE [ROOK] 44 -> 16
9. lépés: FEHER [BISHOP] 6 -> 44
10. lépés: FEKETE [FLAG] 67 -> 69 (ZASZLO CSERE)
11. lépés: FEHER [BISHOP] 43 -> 69 (!!! ZASZLO UTES - GYOZELEM !!!)

```

4.3. ábra. `teszt_szimulacio.txt` tartalma

ki az indítható lépéseket a sűrű kezdőállás ellenére. A futók 10, a bástyák 11 helyre tudnak lépni a kezdőállásból. Mind a futók, mind a bástyák egyből tudnak ellenséges bábút ütni.

4.3. Random játszma szimuláció (End-to-End teszt)

A legösszetettebb ellenőrzési fázist a `RandomGameSimulation()` függvény biztosította, amely egy teljes végponttól-végpontig terjedő (end-to-end) játékmenetet szimulált. A függvény minden körben összegyűjtötte a soron következő csapat összes bábuja által indítható, összesített szabályos lépéskombinációt, majd azokból egy pszeudovéletlen-generátor segítségével kiválasztott és végrehajtott egyet.

A szimulációt egy 300 lépéses biztonsági korláttal láttam el, hogy elkerüljem a végtelen ciklusokat, az eredményeket pedig a `teszt_szimulacio.txt` (4.3) fájlba rögzítettem.

Itt már valós időben kellett frissítenem a játékkeret (`boardState`). Kezelnem kellett az ütéseket, lépések valós idejű validálását, és a memória frissítését. Minden lépést kiirattam, így nyomon lehet követni a meccs alakulását.

4.4. Fizikális ellenőrzés

A szoftveres tesztek eredményességének végső bizonyításához a generált naplófájlok adatait manuálisan is leellenőriztem. Ehhez a projekt rendelkezésére álló, Kiss Márton (1.1) által 3D nyomtatott, fizikális 72 mezős fullerén alakzatot vettem alapul.

A `teszt_szimulacio.txt` (4.3) fájlból vett konkrét mérkőzés lépéssorozatait lépésről lépésre, manuálisan is végigjátszottam a fizikai testen. A fizikális ellenőrzés teljes mértékben igazolta a játék működését: a kalkulált szomszédsági kapcsolatok, a bábuk

mozgásterei és a szimulált random mérkőzések menete a valóságban, a háromdimenziós geometriai testen is tűpontosan, hiba nélkül és a játékszabályok betartásával működtek. Ezzel bizonyítást nyert, hogy a lefejlesztett hálózat architektúrája hibátlanul modellezi a fullerén sakk elméleti rendszerét, és hosszú távon is stabil alapot biztosít a későbbi grafikus felület és a tanítási folyamatokhoz.

5. fejezet

Összefoglalás

A félév során megismerkedtem a háromdimenziós, nem-euklideszi felületeken történő játékmódellezés különböző lehetőségeivel és megalkotásával. Megtanultam milyen lépésekben kell játékot tervezni és fejleszteni.

Sikerült megvalósítanom a 72 mezős fullerén sakkot C++ nyelven. Felépítettem a hálózati architektúrát, integráltam az egyedi szabályokat, és tesztekkel bizonyítottam azok helyességét.

A jövőben tervezek tovább dolgozni a témával. Mivel a 72 mezős csonkított ikozaéder topológiailag fel van építve, a jövőbeli tervek közé tartozik, ugyanerre az alakzatra más játékszabályok integrálása. Ezzel nagyon sok lehetőség nyílik, hogy további játékot fejlesszünk. Továbbá a jelenlegi fullerén sakk gép elleni játszhatósága lenne a cél.

Irodalom

- [1] Władysław Gliński. *First theories of Hexagonal Chess*. London, UK: Hexagonal Chess Publications, 1974. ISBN: 0-904195-01-5.
- [2] A. Mottaghi és Ali R. Ashrafi. „Topological edge properties of C60+12n fullerenes”. *Beilstein Journal of Nanotechnology* 4 (2013), 400–405. old. DOI: **10.3762/bjnano.4.47**.
- [3] David B. Pritchard és John Beasley. *The Classified Encyclopedia of Chess Variants*. 2nd. Harpenden, UK: John Beasley, 2007. ISBN: 978-0-955516-0-1.
- [4] Tamás Réti és tsai. „Fullerének stabilitásának predikciója topológiai invariánsok felhasználásával”. *Fiatal Műszakiak Tudományos Ülésszaka (FMTÜ) XVIII*. Kolozsvár: Erdélyi Múzeum-Egyesület, 2013, 343–350. old.
- [5] Sketchfab. *C60 Fullerene 3D Model*. <https://sketchfab.com/3d-models/c60-fullerene-8fc5315320b1477c9c5f9936dfdac21a>. Elérhető: 2026. május 26. 2026.